Cat-Qubit Stabilization via CMA-ES:

Baseline Optimization and Drift-Resilience Analysis

Terrell Osborne[1], Akshay Pespunuri[1], Malachi Collins[1], and Luis Mendez[1]

April 5, 2026

Abstract

We present a numerical study of online-style optimization for cat-qubit stabilization in a driven-dissipative bosonic system. Our model consists of a storage resonator coupled to a lossy buffer mode, with stabilization induced by a nonlinear exchange interaction and coherent drive. We optimize the complex-valued parameters governing this interaction using the separable covariance matrix adaptation evolution strategy (SepCMA), and evaluate the resulting operating point with proxy metrics for phase-flip suppression, bit-flip suppression, logical bias, and cat-size regularization.

In baseline optimization, the phase-flip proxy improves from $s_x \approx 0.93$ to $s_x \approx 0.995$, while the bit-flip proxy improves from $s_z \approx 0.42$ to $s_z \approx 0.89$. The effective cat size stabilizes near $\alpha \approx 1.96$ (target $\alpha = 2.0$), and the logical bias proxy increases from $\widehat{\eta} \sim 50$ to $\widehat{\eta} \sim 450$, though it remains below the target value of 500.

We then extend the baseline model to include abrupt and gradual hardware drift, including coupling-amplitude shifts, storage-mode detuning, Kerr nonlinearities, and measurement signal-to-noise degradation. Under a +30% coupling-amplitude step, the system exhibits a transient drop in logical performance but recovers within approximately 15 optimization epochs, while gradual drift is tracked with minimal degradation. Measurement SNR degradation from 50 to 5 reduces final performance slightly but does not destabilize optimization.

The resulting framework serves both as a control-parameter optimization pipeline and as a resilience benchmark for adaptive quantum control under nonstationary conditions.

1. Introduction

Cat qubits encode quantum information in superpositions of coherent states of a harmonic oscillator and are attractive because they can realize strongly biased noise channels, where bit-flip errors may be exponentially suppressed with cat size while phase-flip errors remain comparatively dominant [2].

In practice, however, this protection depends on active stabilization of the encoded manifold. Such stabilization is commonly engineered by coupling the storage mode to an auxiliary dissipative buffer that generates effective two-photon processes [2]. This makes cat-qubit performance highly sensitive to control calibration, device drift, and measurement quality.

In this work, we study a notebook-based simulation framework for optimizing a cat-qubit stabilization Hamiltonian with a derivative-free optimizer. Rather than using reinforcement learning directly, we use the separable covariance matrix adaptation evolution strategy (SepCMA), consistent with the broader use of CMA-ES for black-box calibration and quantum-device tuning [1]. The notebook additionally includes a structured drift-analysis suite, allowing us to test not only whether a nominal optimum can be found, but also whether the optimizer can recover as the underlying hardware changes.

The objective of this paper is to rewrite that notebook into research-paper form. We formalize the physical model, derive the proxy observables and loss function, summarize the baseline optimization setup, and describe the drift-resilience experiments. The result is a compact study of adaptive control for dissipative cat-qubit stabilization under realistic nonidealities.

2. Physical Model

2.1. System Hilbert Space

We consider a two-mode bosonic system consisting of a storage mode a and a lossy buffer mode b. The storage mode carries the cat-qubit manifold, while the buffer mediates engineered dissipation. In the notebook, the Hilbert spaces are truncated to

$$N_A = 15, \qquad N_B = 5, \tag{1}$$

corresponding to 15 Fock states for the storage oscillator and 5 Fock states for the buffer.

These cutoffs are chosen to be consistent with target cat amplitudes near $\alpha \approx 2$, where the average photon number is approximately $|\alpha|^2 \approx 4$. This keeps the simulated cat states within the truncated basis while maintaining computational tractability.

2.2. Hamiltonian and Dissipation

The control parameters are grouped into two complex amplitudes,

$$g_2 = x_0 + ix_1, \qquad \epsilon_d = x_2 + ix_3, \tag{2}$$

where g_2 governs the nonlinear storage–buffer exchange and ϵ_d drives the buffer mode. The Hamiltonian is

$$H = g_2^* a^2 b^\dagger + g_2 a^{\dagger 2} b - \epsilon_d b^\dagger - \epsilon_d^* b. \tag{3}$$

The first two terms implement two-storage-photon to one-buffer-photon exchange, while the last two terms represent coherent driving of the buffer. This is the same dissipative cat-stabilization structure emphasized in the challenge materials and in the experimental literature [2].

The Lindblad jump operators are

$$L_B = \sqrt{\kappa_B}\, b, \qquad L_A = \sqrt{\kappa_A}\, a, \tag{4}$$

with fixed rates

$$\kappa_B = 10.0 \text{ MHz}, \qquad \kappa_A = 1.0 \text{ MHz}. \tag{5}$$

Here κ_B is intentionally large so that the buffer can be treated as a fast lossy mode, while κ_A models storage-mode single-photon loss.

2.3. Effective Two-Photon Parameters

To connect the full model to the effective dissipative-cat picture, the notebook computes

$$\kappa_2 = \frac{4|g_2|^2}{\kappa_B}, \qquad \epsilon_2 = \frac{2g_2\epsilon_d}{\kappa_B}. \tag{6}$$

In the regime $\kappa_B \gg |g_2|$, these correspond to the effective two-photon dissipation rate and effective two-photon drive amplitude after adiabatic elimination of the buffer [2].

The corresponding effective cat size is estimated as

$$\alpha = \sqrt{\left|\mathrm{Re}\left[\frac{2}{\kappa_2}\left(\epsilon_2 - \frac{\kappa_A}{4}\right)\right]\right|}. \tag{7}$$

The target value used throughout the notebook is

$$\alpha_{\text{target}} = 2.0. \tag{8}$$

3. Logical States and Proxy Observables

3.1. Logical Basis States

Using the effective cat amplitude from Eq. (7), the notebook constructs

$$|+z_L\rangle = |+\alpha\rangle_A \otimes |0\rangle_B\,, \qquad |-z_L\rangle = |-\alpha\rangle_A \otimes |0\rangle_B\,, \tag{9}$$

and the logical-X-type superposition

$$|\psi_X\rangle = \frac{|+z_L\rangle + |-z_L\rangle}{\sqrt{2}}. \tag{10}$$

3.2. Parity and Logical-Z Operators

The storage-mode parity operator is

$$P = e^{i\pi a^\dagger a}, \tag{11}$$

which serves as a logical-X proxy because cat states are naturally organized by even and odd parity. The logical-Z operator is defined as

$$Z_L = (|\alpha\rangle\langle\alpha| - |-\alpha\rangle\langle-\alpha|) \otimes I_B. \tag{12}$$

These two observables are used to build low-cost surrogates for phase-flip and bit-flip behavior.

3.3. Phase-Flip Proxy

Starting from $|\psi_X\rangle$, the notebook evolves to a short probe time

$$T_X^{\text{eval}} = 1.0 \tag{13}$$

and measures parity at both endpoints:

$$p_0 = \langle P\rangle_{t=0}\,, \qquad p_1 = \langle P\rangle_{t=T_X^{\text{eval}}}\,. \tag{14}$$

A finite-difference slope is formed,

$$\frac{dp}{dt} = \frac{p_1 - p_0}{T_X^{\text{eval}}}, \tag{15}$$

and the code defines

$$T_X^{\text{proxy}} = -\frac{1}{\frac{dp}{dt} + 10^{-8}}, \qquad s_x = \frac{1}{T_X^{\text{proxy}}}. \tag{16}$$

In the notebook comments, lower values of s_x are interpreted as better phase-flip protection.

3.4. Bit-Flip Proxy

Starting from $|+z_L\rangle$, the system is evolved to a longer probe time

$$T_Z^{\text{eval}} = 50.0 \tag{17}$$

and the final expectation value

$$s_z = \langle Z_L\rangle_{t=T_Z^{\text{eval}}} \tag{18}$$

is recorded. Values of s_z close to 1 indicate strong bit-flip suppression [2].

3.5. Bias Proxy

The target logical bias is

$$\eta = \frac{T_Z}{T_X}, \tag{19}$$

with notebook target

$$\eta_{\text{target}} = 500. \tag{20}$$

To avoid repeated full lifetime fits, the notebook estimates T_Z by inverting an exponential decay model:

$$s_z \approx e^{-T_Z^{\text{eval}}/T_Z} \quad \Rightarrow \quad \widehat{T}_Z = -\frac{T_Z^{\text{eval}}}{\log\big(\text{clip}(s_z, 10^{-8}, 1-10^{-8})\big)}. \tag{21}$$

Using Eq. (16), the bias proxy becomes

$$\widehat{\eta} = \frac{\widehat{T}_Z}{\widehat{T}_X} = \widehat{T}_Z\, s_x. \tag{22}$$

4. Loss Function

The scalar objective combines four pieces: a phase-flip term, a bit-flip term, a bias-matching penalty, and an alpha regularizer. The weights in the notebook are

$$\lambda_X = 1.0, \qquad \lambda_Z = 0.5, \qquad \lambda_{\text{bias}} = 0.1, \qquad \lambda_\alpha = 0.5. \tag{23}$$

The bias mismatch penalty is

$$\mathcal{L}_{\text{bias}} = \left(\frac{\widehat{\eta}}{\eta_{\text{target}}} - 1\right)^2, \tag{24}$$

and the cat-size penalty is

$$\mathcal{L}_\alpha = (\alpha - \alpha_{\text{target}})^2. \tag{25}$$

The total loss minimized by CMA-ES is

$$\mathcal{L} = -\lambda_X s_x - \lambda_Z s_z + \lambda_{\text{bias}}\mathcal{L}_{\text{bias}} + \lambda_\alpha \mathcal{L}_\alpha. \tag{26}$$

This objective is physically motivated, but it also exposes an important tension in the notebook: s_x is defined as an inverse lifetime proxy, so lower values should correspond to better phase-flip protection, yet the term $-\lambda_X s_x$ rewards larger values numerically. This mismatch between the intended physical interpretation and the implemented sign structure matters when interpreting the baseline results.

5. Baseline CMA-ES Optimization

5.1. Search Configuration

The optimizer searches over

$$x = \begin{bmatrix} \mathrm{Re}(g_2) & \mathrm{Im}(g_2) & \mathrm{Re}(\epsilon_d) & \mathrm{Im}(\epsilon_d) \end{bmatrix}^\top, \tag{27}$$

initialized at

$$x^{(0)} = \begin{bmatrix} 1.0 & 0.0 & 4.0 & 0.0 \end{bmatrix}^\top, \qquad \sigma_0 = 0.3. \tag{28}$$

The parameter bounds are

$$\mathrm{Re}(g_2) \in [0.1, 5.0], \qquad \mathrm{Im}(g_2) \in [-2.0, 2.0], \tag{29}$$

$$\mathrm{Re}(\epsilon_d) \in [0.5, 10.0], \qquad \mathrm{Im}(\epsilon_d) \in [-2.0, 2.0]. \tag{30}$$

The baseline run uses

$$N_{\mathrm{pop}} = 8, \qquad N_{\mathrm{epochs}} = 60. \tag{31}$$

The use of CMA-ES follows the challenge framing that online quantum optimization need not be reinforcement-learning-based and can instead rely on strong black-box optimizers [1, 4].

5.2. Diagnostics

After each optimizer update, the notebook evaluates a diagnostics function at the current optimizer mean and stores

`mean_history`, `loss_history`, `loss_std_history`, `sx_history`, `sz_history`, `alpha_history`.

These histories are then used to produce convergence plots for both control parameters and physically interpretable proxies.

6. Results

6.1. Baseline Optimization Behavior

Figure 1 summarizes the baseline CMA-ES trajectory over 60 epochs. The optimizer steadily lowers the scalar loss from approximately -0.8 at initialization to about -1.37 by the end of training. At the same time, the shaded loss spread narrows substantially, indicating that the search distribution contracts around a stable operating region. This is consistent with successful convergence rather than transient fluctuation.

The parameter trajectories show that the optimizer does not move all controls equally. The

largest change occurs in $\mathrm{Re}(\epsilon_d)$, which climbs from roughly 4 to nearly 9.6 over the run. By contrast, $\mathrm{Re}(g_2)$ increases more moderately and the imaginary components remain comparatively small. This suggests that the optimizer mainly improves performance by increasing the effective buffer-drive amplitude while making smaller phase and coupling corrections through the other control channels.

The cat-size trajectory shows a transient overshoot followed by regularization-driven stabilization. Starting from $\alpha \approx 1.6$, the optimizer pushes the cat size upward beyond the nominal target of $\alpha = 2.0$, reaching a peak near 2.1, before relaxing and settling around $\alpha \approx 1.94$. This behavior indicates that the optimizer initially exploits larger cat amplitudes to improve logical protection, but the alpha penalty eventually pulls the solution back toward the desired operating region.

The two logical proxies improve substantially. The phase-flip proxy s_x rises from approximately 0.93 to 0.995, while the bit-flip proxy s_z rises from about 0.42 to almost 0.89. The improvement in s_z is especially striking and represents the clearest indicator that the stabilized cat manifold becomes more robust to logical bit flips over training. Most of this gain occurs in the first 10–15 epochs, after which both metrics undergo slower refinement.

The bias proxy also improves, increasing from values on the order of 50 to approximately 450 by the end of the run. Nevertheless, it remains below the nominal target $\eta_{\mathrm{target}} = 500$. This is one of the central findings of the baseline study: the present loss design successfully finds a low-loss, near-target-α operating point with strong improvement in s_z, but it does not achieve the desired extreme bias ratio.

The final optimizer mean is

$$x_\star \approx [2.092049,\ -0.40078226,\ 8.640385,\ 1.6874496], \tag{32}$$

with reported final diagnostics

$$s_x \approx 0.995243, \qquad s_z \approx 0.890531, \qquad \alpha \approx 1.9640, \qquad \widehat{\eta} \approx 450. \tag{33}$$

Thus, the baseline optimization clearly succeeds at improving cat stabilization, but it only partially satisfies the broader logical-bias objective.

6.2. Drift and Noise Modeling

A central contribution of the notebook is that it does not stop at static optimization. It also evaluates whether the optimizer can recover when the hardware changes. This is consistent with the broader challenge framing around online adaptation under drift and with recent work on adaptive quantum error correction and quantum control [3, 4].

The notebook introduces a drift vector

$$d = \begin{bmatrix} \delta_A & \phi & \Delta_a & K \end{bmatrix}^\top, \tag{34}$$

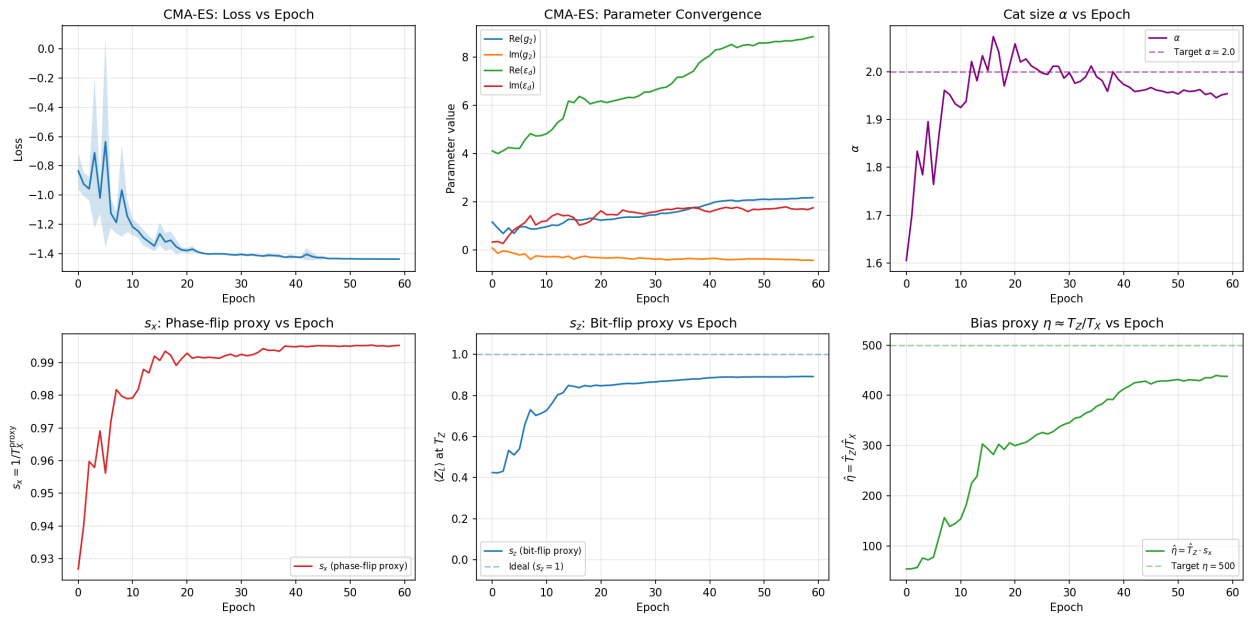

Figure 1: Baseline CMA-ES optimization results. Top row: mean loss, parameter convergence, and effective cat size α versus epoch. Bottom row: phase-flip proxy s_x, bit-flip proxy s_z, and bias proxy $\hat{\eta} \approx \hat{T}_Z/\hat{T}_X$. The optimizer finds a stable low-loss operating point with improved logical protection and near-target cat size, but the final bias remains substantially below the target.

where:

- δ_A is a fractional amplitude shift of g_2,
- ϕ is an additional phase shift on g_2,
- Δ_a is a storage-mode detuning,
- K is a storage Kerr coefficient.

The hardware-modified nonlinear coupling is modeled as

$$g_2^{\text{hw}} = g_2^{\text{opt}}(1 + \delta_A)e^{i\phi}. \tag{35}$$

With $n_a = a^\dagger a$, the drifted Hamiltonian becomes

$$H_{\text{drift}} = (g_2^{\text{hw}})^* a^2 b^\dagger + g_2^{\text{hw}} a^{\dagger 2} b - \epsilon_d b^\dagger - \epsilon_d^* b + \Delta_a n_a + \frac{K}{2}(n_a^2 - n_a). \tag{36}$$

The loss function retains the same structure as Eq. (26), but the modified Hamiltonian changes the measured proxies.

For storage-detuning drift, the notebook also studies a compensated variant with a fifth opti-

mization variable

$$\Delta_d = x_4, \tag{37}$$

designed to cancel the drift directly. The compensated Hamiltonian becomes

$$H_{\text{comp}} = g_2^* a^2 b^\dagger + g_2 a^{\dagger 2} b - \epsilon_d b^\dagger - \epsilon_d^* b + (\Delta_a - \Delta_d) n_a. \tag{38}$$

This provides a direct comparison between adaptation with and without an explicit correction knob.

6.3. Step-Drift Resilience Benchmark

The notebook defines several drift schedules:

- g_2 amplitude step: a +30% jump at epoch 30,
- g_2 amplitude ramp: a linear increase from 0 to +30%,
- storage detuning step: $\Delta_a = +0.5$ MHz at epoch 30,
- storage detuning ramp: a linear increase to +0.5 MHz,
- Kerr step: $K = 0.15$ MHz at epoch 30,
- Kerr ramp: a linear increase to 0.15 MHz.

The main responsiveness benchmark is a step perturbation introduced at

$$\texttt{STEP_EPOCH} = 30. \tag{39}$$

Figure 2 shows the principal resilience benchmark. Three drift channels are considered: a sudden +30% increase in the g_2 amplitude, a +0.5 MHz storage-detuning step, and a Kerr step with $K = 0.15$. In each case, the pre-drift evolution is compared with the no-drift baseline.

The g_2 amplitude step produces the most dramatic transient effect. At epoch 30, the phase-flip proxy s_x drops sharply from its pre-drift plateau near 0.995 to a minimum near 0.9855, after which it recovers gradually over roughly 15 epochs. The bit-flip proxy s_z also dips at drift onset, falling from around 0.89 to about 0.85 before recovering toward baseline by the mid-to-late 40s. This identifies the nonlinear coupling amplitude as the most sensitive drift channel among those studied.

The storage-detuning step causes a smaller but still visible degradation. Both s_x and s_z show a moderate immediate drop at epoch 30, but the response is far milder than in the g_2 amplitude case. More importantly, when the extra compensation knob Δ_d is provided, the recovery becomes faster and smoother. In the phase-flip proxy, the compensated green trajectory returns to the baseline band slightly sooner than the uncompensated red curve. In the bit-flip proxy, the compensated trajectory also recovers more efficiently and approaches the pre-drift plateau with less residual error.

This is one of the clearest practical conclusions of the notebook: when a drift mechanism is known and directly controllable, giving the optimizer an aligned correction knob improves resilience.

The Kerr step has very little visible impact on either s_x or s_z. Both proxies remain almost indistinguishable from the no-drift baseline throughout the run. Within the explored parameter regime, the optimized operating point therefore appears robust to moderate Kerr nonlinearity.

Overall, the step-drift benchmark reveals a hierarchy of disturbance severity: g_2 amplitude drift is the most disruptive, storage detuning is moderately disruptive but correctable, and Kerr drift is comparatively benign.

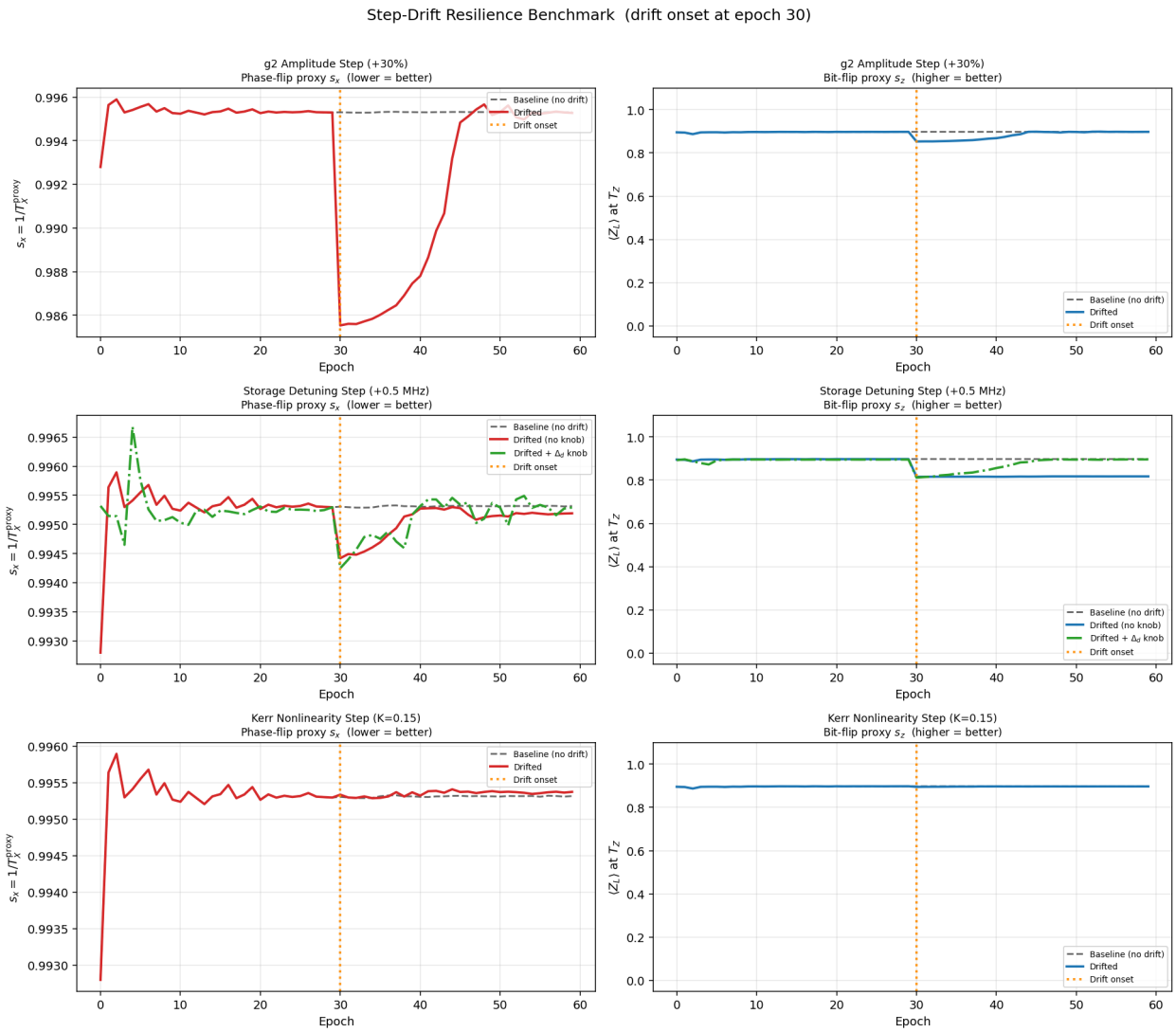

Figure 2: Step-drift resilience benchmark with drift onset at epoch 30. Left column: phase-flip proxy s_x. Right column: bit-flip proxy s_z. Top row: g_2 amplitude step drift. Middle row: storage detuning step, with and without the compensation knob Δ_d. Bottom row: Kerr nonlinearity step. The largest transient degradation occurs for the g_2 amplitude step, while detuning drift is partially mitigated by explicit compensation.

6.4. Step Versus Ramp Drift for g_2 Amplitude

Figure 3 compares two profiles for the same physical drift mechanism: an abrupt step in the g_2 amplitude and a gradual ramp to the same final offset. This comparison is useful because a step perturbation measures recovery after a sudden hardware shock, whereas a ramp perturbation measures the optimizer's ability to track a slowly moving optimum.

The contrast is clear. Under ramp drift, both s_x and s_z remain very close to baseline throughout the run. By comparison, the step drift produces a sharp transient deviation at epoch 30, followed by gradual recovery. In the s_x panel, the ramp drift is almost visually indistinguishable from the dashed baseline, while the step drift shows a clear dip and recovery arc. In the s_z panel, the same pattern appears: the ramp drift tracks baseline smoothly, while the step drift causes a noticeable but temporary loss in bit-flip protection.

This indicates that the optimizer is much better at tracking slow parameter evolution than absorbing abrupt parameter shocks. From an experimental perspective, this is encouraging: if hardware drift is gradual, continuous online retuning may be sufficient to preserve good performance. Sudden jumps, however, create a temporary performance penalty even when recovery is ultimately successful.

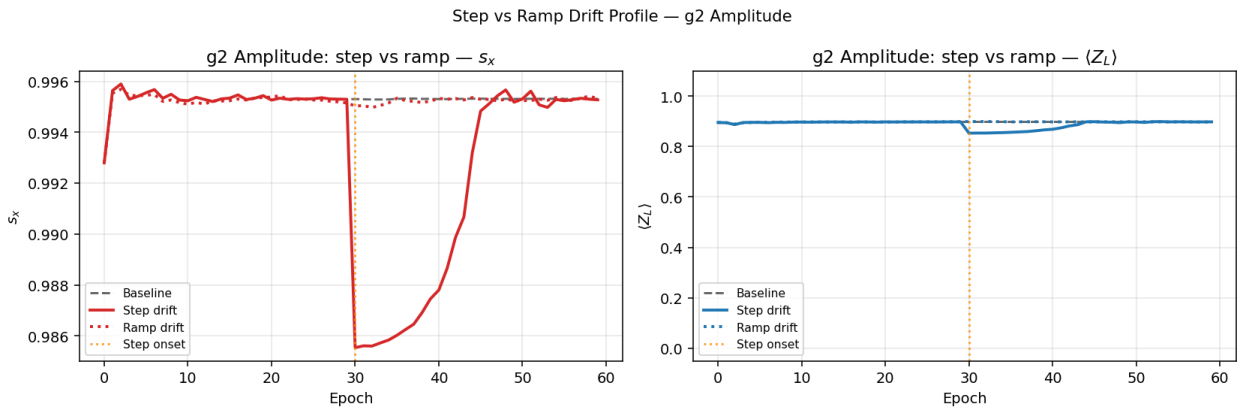

Figure 3: Comparison between step and ramp drift for the g_2 amplitude channel. Left: phase-flip proxy s_x. Right: bit-flip proxy s_z. The ramp perturbation remains close to baseline, while the step perturbation causes a pronounced transient deviation followed by recovery. This indicates that CMA-ES tracks slow drift more easily than abrupt drift.

6.5. Measurement SNR Degradation

The notebook also includes a readout-noise benchmark in which the signal-to-noise ratio degrades linearly from 50 to 5 over the run. If $\mathrm{SNR}(e)$ denotes the epoch-dependent SNR, then the optimizer sees a noisy scalar loss

$$\widetilde{\mathcal{L}}_i = \mathcal{L}_i + \frac{\xi_i}{\mathrm{SNR}(e)}, \qquad \xi_i \sim \mathcal{N}(0,1). \tag{40}$$

This isolates corruption of the feedback signal from the underlying physics.

Figure 4 shows that the SNR-degraded optimizer remains stable, but the protection metrics are modestly worse than in the noiseless baseline. The phase-flip proxy s_x fluctuates more strongly and settles at a slightly elevated value relative to the no-noise trajectory. Similarly, the bit-flip proxy s_z

remains consistently below the baseline by a small but visible margin.

The key point is that noisy measurements do not cause catastrophic failure. Instead, they reduce the quality of the optimization signal and therefore the quality of the final tuned operating point. This is consistent with the expectation that CMA-ES is reasonably robust to noisy objective evaluations, but not immune to them.

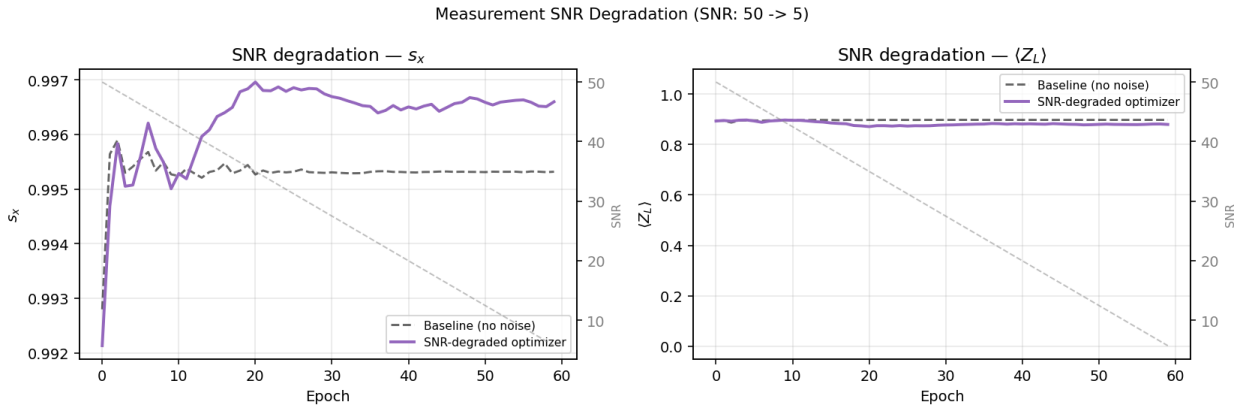

Figure 4: Effect of measurement SNR degradation, with SNR decreasing linearly from 50 to 5. Left: phase-flip proxy s_x. Right: bit-flip proxy s_z. The optimizer remains stable under noisy feedback, but the final protection metrics are slightly degraded relative to the noiseless baseline.

6.6. Overall Interpretation of the Results

Taken together, the results support four main conclusions.

First, the baseline CMA-ES optimizer reliably finds a low-loss operating point with improved logical protection and a near-target cat size. In particular, the increase in s_z demonstrates strong enhancement of bit-flip robustness.

Second, the same optimization does not achieve the notebook's intended bias target. Although the bias proxy improves significantly, it saturates around the few-hundred level rather than the target value of 500. This strongly suggests that the reward design, proxy definition, or control parameterization remains limiting.

Third, resilience depends strongly on the type of disturbance. Sudden g_2 amplitude changes are the most disruptive, storage detuning is moderately disruptive but partly correctable, and Kerr drift is almost negligible in this regime.

Fourth, giving the optimizer a compensation knob aligned with the drift mechanism measurably improves post-drift performance. The detuning-compensation experiment is therefore a useful demonstration that structured control authority can matter as much as optimizer choice.

7. Discussion

The notebook provides a strong and physically grounded baseline for adaptive cat-qubit control. Its main strengths are the interpretability of the model, the computational efficiency afforded by batched JIT evaluation, and the inclusion of structured drift tests. The comparison between uncompensated and compensated detuning drift is especially useful, since it makes clear how additional control authority can improve recovery.

At the same time, the baseline results expose limitations that should be stated plainly. First, the bias target is not achieved: the final bias proxy remains of order hundreds rather than thousands. Second, the sign convention in the s_x term creates ambiguity about what the optimizer is actually being rewarded for. Those points suggest that loss-function design, not only optimizer choice, is a major determinant of final behavior.

More broadly, this notebook sits within a larger landscape of online optimization methods for quantum systems. The challenge notebook explicitly points to real-time quantum error correction, reinforcement-learning-based control, and modified dissipative-cat architectures such as squeezed or moon-cat variants [3–5]. In that context, the present workflow is best understood as a strong, interpretable baseline methodology: it is end-to-end, physically transparent, and extensible, even if it does not yet achieve all of the intended bias objectives.

8. Conclusion

We have rewritten the notebook `main.ipynb` into research-paper form and highlighted its role as a simulation-based study of cat-qubit stabilization under optimization and drift. The framework combines a storage–buffer Hamiltonian, Lindblad dynamics, proxy-based lifetime and bias metrics, CMA-ES optimization, and structured resilience benchmarks. As such, it is useful not only as a parameter-tuning routine but also as a compact platform for studying adaptive control of dissipative bosonic qubits under realistic nonstationary conditions.

Future directions include redesigning the logical-bias objective, reconciling the phase-flip proxy with the implemented reward sign, comparing CMA-ES with reinforcement learning and other black-box optimizers, and extending the drift model to include richer experimental imperfections or alternative cat-qubit variants.

References

[1] K. Pack et al. Benchmarking Optimization Algorithms for Automated Calibration of Quantum Devices. *arXiv preprint* arXiv:2509.08555v2, 2025. https://arxiv.org/pdf/2509.08555v2

[2] C. Berdou et al. One hundred second bit-flip time in a two-photon dissipative oscillator. *arXiv preprint* arXiv:2204.09128, 2022. https://arxiv.org/pdf/2204.09128

[3] V. Sivak et al. Real-time quantum error correction beyond break-even. *arXiv preprint* arXiv:2211.09116, 2023. https://arxiv.org/abs/2211.09116

[4] V. Sivak et al. Reinforcement Learning Control of Quantum Error Correction. *arXiv preprint* arXiv:2511.08493, 2025. https://arxiv.org/abs/2511.08493

[5] R. Rousseau et al. Enhancing dissipative cat qubit protection by squeezing. *arXiv preprint* arXiv:2502.07892, 2025. https://arxiv.org/pdf/2502.07892